Lab Four Arcade Machine Lab Report

Jack Ewing, Tyler Barton
10/4/2026

Original FPGA Project Proposal

This project proposal was submitted on Sept. 23rd.

Project Description

The system is an FPGA arcade cabinet simulator that holds 3 games, Snake, Pong, and Flappy Bird. The system will manage everything, including state machine transitions, menu navigation, collision detection, graphic rendering, and score systems.

Inputs: Onboard buttons (btnU, btnD, btnL, btnR, btnC), 100MHz clock,

Outputs: Seven segment display (Score counter), VGA outputs 640x480 @ 60 Hz to an external monitor

Facets of Interaction:

1. At launch, the user is prompted by a screen to choose which of the three games they will pick. The user can navigate this menu using btnU / btnD, and choosing with btnC.
2. During Flappy Bird, the user can move the bird up by pressing btnU.
During Snake, the user can maneuver the snake by using the 4 directional buttons (up, down, left, right).
3. During Pong, the user can manipulate the board to hit the pong using the up and down buttons.
4. During all games, pressing the btnC will return the user to the menu.

FPGA Justification

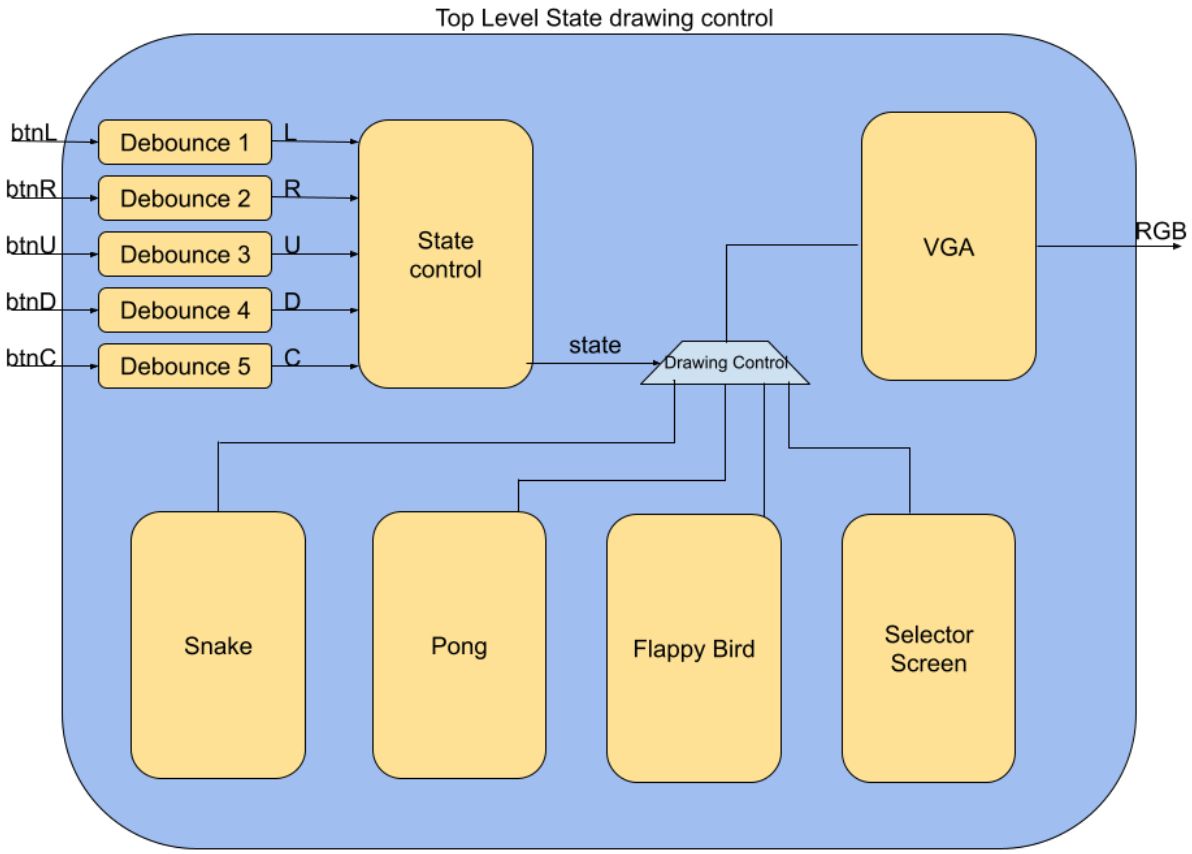
Application-Specific Integrated Circuits (ASIC) for custom digital hardware allows for low production cost when manufacturing thousands or millions of boards, but it is permanent and is unable to be used well to prototype a system. Utilizing an FPGA like the Basys 3 provides an ideal prototyping environment, perfect for simulating the early stages of an arcade machine before mass production. By implementing the entire logic architecture in reconfigurable hardware, it can be validated & debugged immediately, without having to go through the ASIC design process again.

Unlike software systems that process things sequentially, FPGAs utilizes parallel hardware, meaning game physics, menu navigation, and VGA signal gen process every clock cycle. Because hardware logic updates states instantly on clock edges, frame stuttering is reduced to zero, and input lag due to software is as well. The only sources of input lag are the button debounce and the refresh rate of the monitor.

Usage

The device begins on the selection screen with three options, the user can choose their selection by using the left and right buttons to pick which game to play. To select the game the user hits the middle button and then the game is loaded. The user also controls the game by using the left, right, up, and down button and they perform the corresponding game action.

Top Level Sub Modules



Debounce

inputs/outputs

name	size	in/out
clk	1’b	in
din	1’b	in

dout	1’b	out

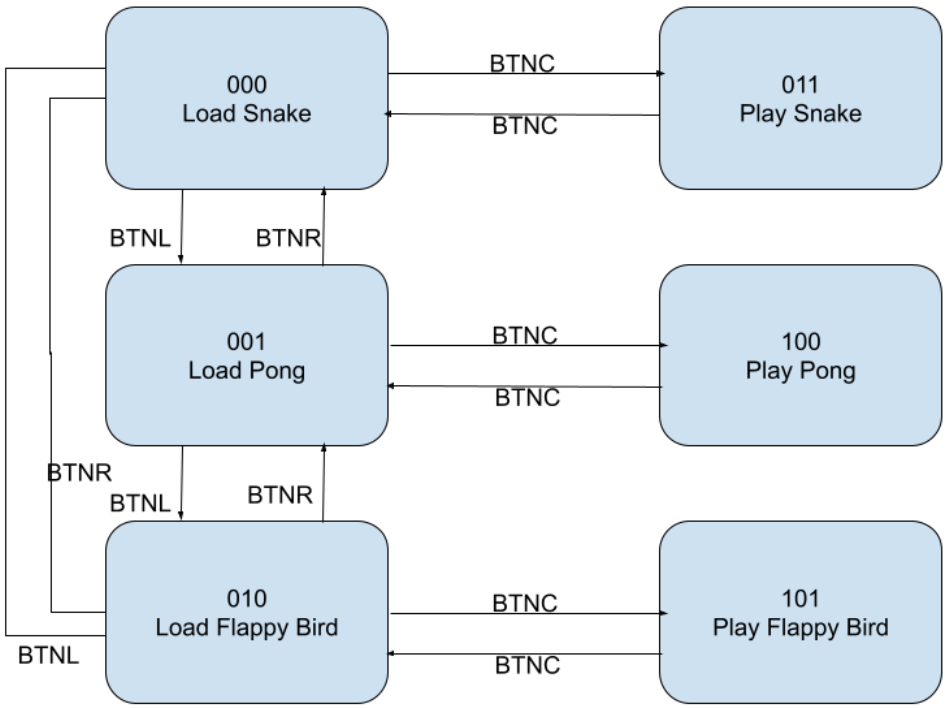
Code description
This module is used on all button inputs to avoid having issues with buttons being counted twice. This makes changing states and controlling the games much easier and playing the game actually possible.

Debounce (Verification)



This testbench waveform diagram shows the debouncing module doing its job and properly preventing the button from giving multiple inputs making the device much easier to work with and the games possible to play.

State Control



inputs/outputs

name	size	in/out
clk	1’b	in
PressC	1’b	in
PressL	1’b	in
PressR	1’b	in
PressU	1’b	in
PressD	1’b	in
state	3’b	out

Code description
This module controls the state using a simple case statement updated on the positive clock edge. This module uses the debounced button input to avoid running into errors jumping states very fast.

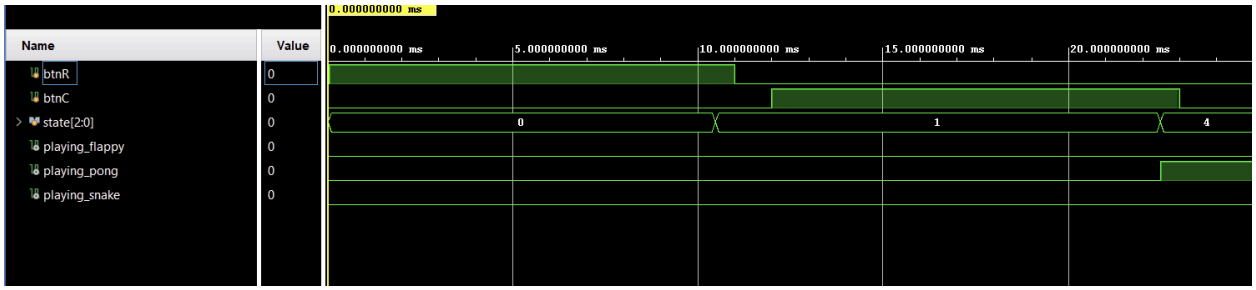
Drawing Control

inputs/outputs

name	size	in/out
video_on	1’b	in
state	3’b	in
rgb	12’b	out

Code description
The selection_screen, snake, and pong modules are always deciding what to draw on the screen; this module just picks which of the options to display. Because of the way a HDL implements the design, having lots of calculations running in parallel is not a problem so this can be done without compromising performance.

Top Level (Verification)



This testbench shows the movement between the different states of the state machine and how that can be used to select a game, in this case that game is pong and by pressing the left button to move to game_two_select (pong selection) then hitting the center button to select the state machine will properly select and play the game.

Pong Sub Modules

Frame clock

inputs/outputs

name	size	in/out
clk	1’b	in
frame_tick	1’b	out

Code description
The uses the rising edge of the clock to create a counter and increment an internal register, when the register reaches a certain point it will create a clock pulse at a much lower frequency.

Paddle movement

inputs/outputs

name	size	in/out
active_state	1’b	in
frame_tick	1’b	in
btnU	1’b	in
btnD	1’b	in
player_y	10’b	out

Code description
This module updates the position of the paddle based on user input either btnU or btnD. It will only update the paddle position until it hits the edge of the screen then it will stop moving in that direction to keep the paddle on the screen.

Comp Tracking

inputs/outputs

name	size	in/out
active_state	1’b	in
frame_tick	1’b	in
ball_center_y	10’b	in
comp_center_y	10’b	in
comp_y	10’b	out

Code description
Updates the position of the computer's paddle until the paddle is at the same y level as the ball so that the computer can hit the ball back. This updates the computer paddle at the same speed as the player paddle, for future improvements it would be a good idea to have it calculate the bounce of the ball because it can currently be tricked by bouncing the ball off of the wall.

Ball Control

inputs/outputs

name	size	in/out
active_state	1’b	ut
frame_tick	1’b	in
player_x_min	10’b	in
player_x_max	10’b	in
player_y	10’b	in
comp_x_min	10’b	in
comp_x_max	10’b	in
comp_y	10’b	in
ball_x	10’b	out
ball_y	10’b	out

Code description
Controls the movement of the ball by tracking its position and velocity and calculating collisions using the position of both paddles and checking to see if the ball will hit it on the next tick, if it

will hit then the ball’s x velocity will be flipped. If the ball bounces off the roof or floor of the screen then the ball's velocity will flip.

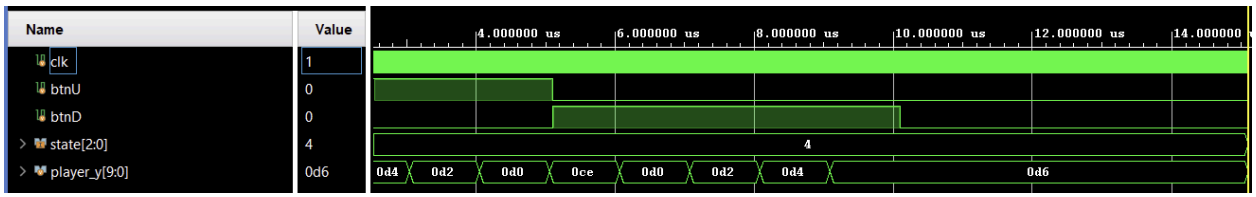
Drawing

inputs/outputs

name	size	in/out
x	10’b	in
y	10’b	in
player_x_min	1’b	in
player_x_max	1’b	in
player_y	10’b	in
comp_x_min	10’b	in
comp_x_max	10’b	in
ball_x	10’b	in
ball_y	10’b	in
ball_size	10’b	in
line_x_min	10’b	in
line_x_max	10’b	in
rgb	12’b	out

Code description
Based on the given pixel it will decide the color of the pixel based on the position of all the elements to correctly draw the screen when needed.

Pong (Verification)



In the testbench for pong, two scenarios were checked, btnU going high then low, and btnD subsequently going high then low. These were the most important values to check, as these buttons are what the player uses to interact with the game, or in other words, move the paddle up and down. While btnU was pressed, the player y value decreased as the paddle went upscreen, and the opposite happened as btnD was pressed, proving validity.

Snake Sub Modules

Frame clock

inputs/outputs

name	size	in/out
clk	1’b	in
frame_tick	1’b	out

Code description
The uses the rising edge of the clock to create a counter and increment an internal register, when the register reaches a certain point it will create a clock pulse at a much lower frequency. So that the snake is not moving at 100mhz and impossible to play.

Pseudo-Random number generator

inputs/outputs

name	size	in/out
clk	1’b	in
lfsr	16’b	out

Code description
The code uses a seed 16'hACE1 to generate a Pseudo-Random number for the spawn of food items for the game. This is not truly random so frame perfect inputs will produce the same result every time however running at this speed in this context it is not a problem.

Direction control

inputs/outputs

name	size	in/out
clk	1’b	in
btnU	1’b	in
btnD	1’b	in
btnL	1’b	in
btnR	1’b	in
tick	1’b	in
dir	3’b	out

Code description
Updates the direction of the snake based on button inputs. This module also prevents the snake from making 180deg turns by keeping track of the previous direction and comparing it to the input direction.

```
// Store requested direction immediately.
        // Don't allow 180 degree turns.
        if (btnU && dir != 2'd2)
            next_dir <= 2'd0;
        else if (btnR && dir != 2'd3)
            next_dir <= 2'd1;
        else if (btnD && dir != 2'd0)
            next_dir <= 2'd2;
        else if (btnL && dir != 2'd1)
            next_dir <= 2'd3;

        // Apply direction on next movement.
        if (tick)
            dir <= next_dir;
```

Snake control

inputs/outputs

name	size	in/out
clk	1’b	in
body_x	8’b	in
body_y	8’b	in
occupied	100’b	in
head_x	8’b	out
head_y	8’b	out

Code description
This module does several things, it first calculates if it will hit a food square so that it can grow the tail. The next thing it checks is to see if a move will run the snake into itself or the wall. After all the collision calculations are completed the snake will be moved forward and the tail will be pulled forward following the head of the snake. At the end of the module it will check if the food was eaten so that it can spawn a new food into a clear square.

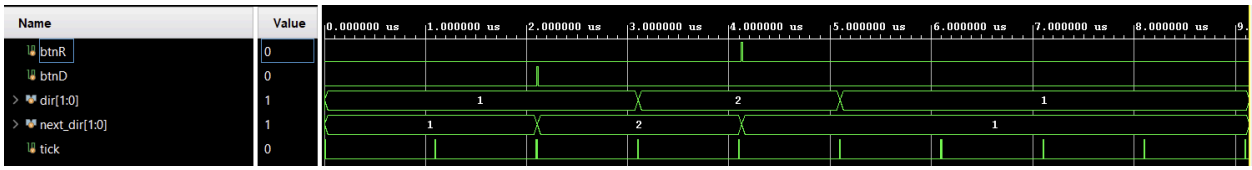
Drawing

inputs/outputs

name	size	in/out
x	10’b	in
y	10’b	in
is_snake	1’b	in
is_food	1’b	in
is_line	1’b	in
rgb	12’b	out

Code description
Based on the given pixel it will decide the color of the pixel based on the position of all the elements to correctly draw the screen when needed. For this game it will only render in the left 480 pixels because that can easily map to a 10 by 10 grid where each square is 48pxls wide. This will draw the food, snake, and map.

Snake (Verification)



This waveform is the verification of the direction control for the snake, as you can see it will decide a direction whenever an input is made based on what button was pressed and the current direction so that the snake never does a full 180deg turn.

Selection Screen

Function draw_box

inputs/outputs

name	size	in/out
x	10’b	in
y	10’b	in
centerX	10’b	in
centerY	10’b	in
width	10’b	in
height	10’b	in
inside	1’b	out

Code description
A function used to determine if the given x and y are inside the defined box. This is used to draw all of the boxes needed for displaying the selection boxes. This function is needed to reduce rewriting code and make the module much easier to read.

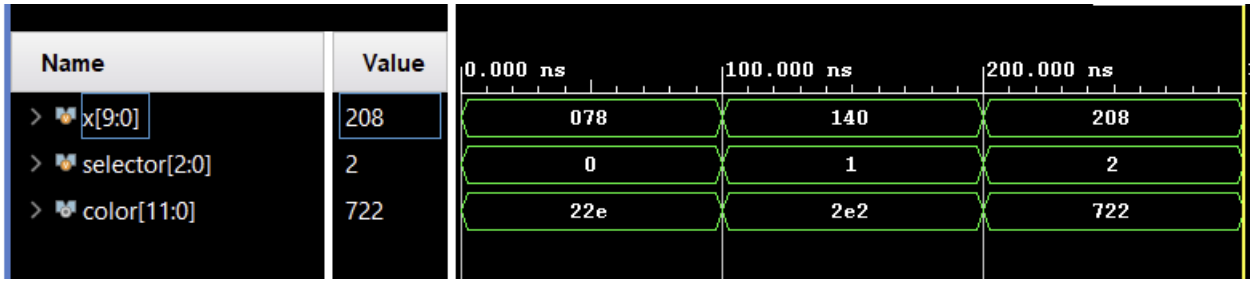
Drawing

inputs/outputs

name	size	in/out
x	10’b	in
y	10’b	in
state	3’b	in
rgb	12’b	out

Code description
Draws the three different boxes for selection and the highlight box based on the state. The boxes have defined positions so that they are aligned to the vertical center and offset 200px on the horizontal axis. The highlighted box is controlled by a case statement based on the state register to highlight one of three positions.

Selection Screen (Verification)



This diagram is using the selector module to control which box is highlighted so that we can use the same module for all three selection states. The color is outputting the correct color for that x screen position.

Flappy Bird Sub Modules

Frame clock

inputs/outputs

name	size	in/out

clk	1’b	in
frame_tick	1’b	out

Code description
The uses the rising edge of the clock to create a counter and increment an internal register, when the register reaches a certain point it will create a clock pulse at a much lower frequency. For this game it is running on 60hz

Collision

inputs/outputs

name	size	in/out
bird_y	1’b	in
pipe_gap	[0:NUM_PIPES-1]	in
hit	1’b	out

Code description
Code to determine if the bird is going to hit the pipe or not. It checks this by getting the bird height and checking it against the ground in the hit box of each pipe, it will update the hit register so that the game logic can “kill” the bird if it makes contact.

```
hit = 1'b0;
if (bird_y < 0 || bird_y + BIRD_S >= GROUND_Y)
    hit = 1'b1;
for (k = 0; k < NUM_PIPES; k = k + 1) begin
    if ((BIRD_X + BIRD_S > pipe_x[k]) && (BIRD_X < pipe_x[k] + PIPE_W) &&
        ((bird_y < pipe_gap[k]) || (bird_y + BIRD_S > pipe_gap[k] + GAP)))
        hit = 1'b1;
end
```

Game logic

inputs/outputs

name	size	in/out
clk	1’b	in

btnU	1’b	in
pipe_x	[0:NUM_PIPES-1]	in
passed	[NUM_PIPES-1:0]	in
bird_y	12’b	out

Code description
Handles moving the pipes across the screen, flapping the bird and keeping track of the score. To flap the bird btnU is brought high and saved until the next frame. It will check the hit register to see if the bird made contact with the object and needs to be put into the dead state. Then it will update the position of the drawing by changing the bird y based on vertical velocity and move the pipes left based on the local parameter SPEED, after all the new positions are calculated if the left pipe is off the screen it will be recycled back to the right and given a new height so the player can keep playing without running out of pipes.

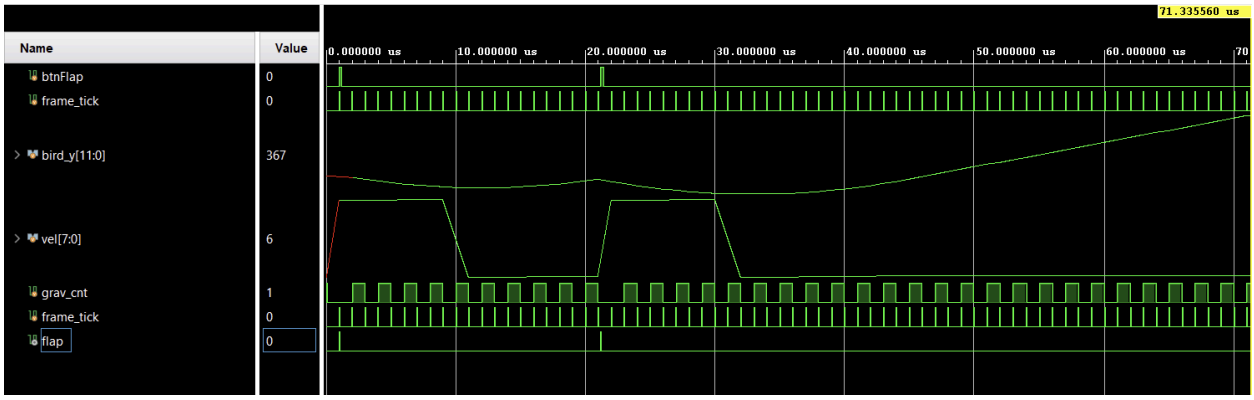
Drawing

inputs/outputs

name	size	in/out
x	10’b	in
y	10’b	in
pipe_x	[0:NUM_PIPES-1]	in
pipe_gap	[0:NUM_PIPES-1]	in
xs	12’b	in
ys	12’b	in
is_score	1’b	in
rgb	12’b	out

Code description
Draws the pipes, scoreboard, and the bird itself, This is the most complicated draw function because it uses lots of different elements for the different parts of the bird and highlighting edges. For a barebones implementation everything can just be a square. This is where a lot of the code complexity for the module comes from.

Flappy Bird (Verification)



During Flappy Bird, the player can interact with the game in one way, by pressing the btnU. Completing this action will cause the bird to launch upwards. In the waveform, as soon as the (flap) flag goes high, velocity immediately spikes, causing the subsequent (bird_y) values to fall as the bird goes up the screen, then as velocity is lost the bird falls back down the screen, resulting in increasing (bird_y) values.

Features and Ideas

When playtesting using the buttons for input was found to be problematic with users sometimes going back to the main menu when they didn't mean to, or making the wrong input specifically in snake. Given more time we would like to add the seven segment display to display score because currently pong does not have a score display to show how well you are playing.

Team Contributions

For this project we split the work based on modules. Tyler wrote the pong module. Jack wrote the top level module, snake module, flappy bird module, and selection screen module. Then we both split the lab report with Jack primarily writing the module descriptions and Tyler primarily making the waveform diagrams.